

**МИНИСТЕРСТВО НАУКИ И ВЫСШЕГО ОБРАЗОВАНИЯ
РОССИЙСКОЙ ФЕДЕРАЦИИ
ФЕДЕРАЛЬНОЕ ГОСУДАРСТВЕННОЕ АВТОНОМНОЕ
ОБРАЗОВАТЕЛЬНОЕ УЧРЕЖДЕНИЕ ВЫСШЕГО ОБРАЗОВАНИЯ
«НАЦИОНАЛЬНЫЙ ИССЛЕДОВАТЕЛЬСКИЙ
ТЕХНОЛОГИЧЕСКИЙ УНИВЕРСИТЕТ «МИСИС»**

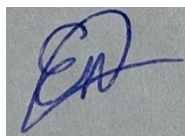
*ИНСТИТУТ Информационных компьютерных наук (ИКН)
КАФЕДРА Автоматизированных систем управления (ИКТ)
НАПРАВЛЕНИЕ 09.03.02 ГРУППА БИВТ-24-8*

КУРСОВОЙ ПРОЕКТ

*ПО КУРСУ: «Клиент-серверные приложения и сетевые
технологии»*

ТЕМА: «Разработка клиент-серверного приложения»

Студент Нечепорук Е.С.



Руководитель проекта Микитенко И.И.

Оценка выполнения курсового проекта _____

Москва 2026 г.

Содержание

Введение.....	3
Задание.....	4
Описание предметной области.....	5
Постановка задачи.....	6
Проектирование архитектуры приложения.....	7
Архитектура приложения.....	8
Проектирование базы данных.....	9
Структура базы данных.....	9
Представления.....	9
Функции, процедуры и триггеры.....	10
Хранимые процедуры.....	11
Реализация серверной части.....	13
Выбор технологий.....	13
Основные эндпоинты API.....	13
Реализация клиентской части.....	16
Структура клиентского приложения.....	16
Функции взаимодействия с API.....	17
Тестирование приложения.....	19
Заключение.....	23
Список литературы.....	24

Введение

Целью выполнения курсового проекта является закрепление знаний, полученных при изучении курса «Клиент-серверные приложения и сетевые технологии», а также приобретение практических навыков разработки полнофункционального веб-приложения с использованием современных технологий.

В рамках проекта разработано клиент-серверное приложение для интернет-магазина велосипедов. Приложение реализует функционал просмотра каталога товаров, фильтрации, добавления в корзину, оформления заказов и просмотра истории покупок.

Задание

Задание: разработать клиент-серверное приложение для интернет-магазина велосипедов с использованием следующих технологий:

- база данных PostgreSQL;
- серверная часть на Python с использованием фреймворка Flask;
- клиентская часть на HTML, CSS и JavaScript.

Вариант №15

Таблица 1 - ТЗ

№	Функция	Описание
1	Регистрация	Создание нового пользователя
2	Авторизация	Вход в систему
3	Каталог товаров	Отображение списка товаров с фильтрацией
4	Корзина	Добавление, удаление, изменение количества товаров
5	Оформление заказа	Создание заказа с адресом доставки
6	История заказов	Просмотр всех заказов пользователя

Описание предметной области

Предметной областью данной работы является интернет-магазин по продаже велосипедов. Основные сущности предметной области включают: пользователей, товары, заказы и позиции заказа.

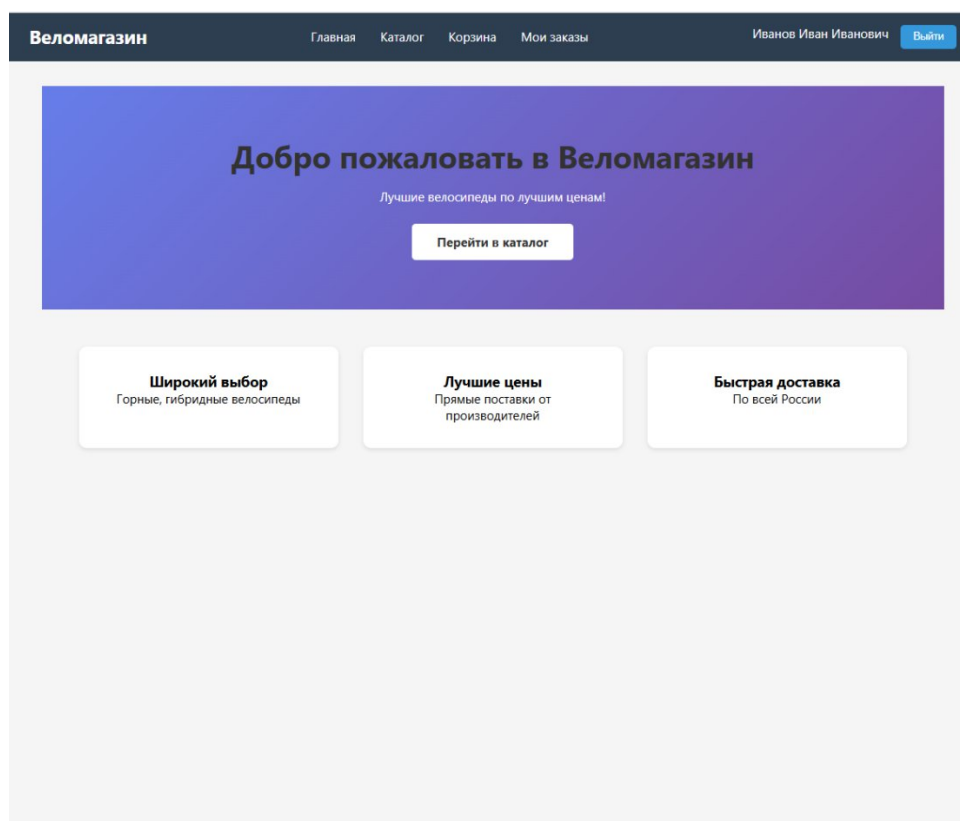


Рисунок 1 — Главная страница интернет-магазина

Основные проблемы, существующие в данной предметной области:

- необходимость обработки большого количества товаров и заказов;
- требование к надёжному хранению данных;
- обеспечение безопасности пользовательских данных;
- необходимость поддержки одновременной работы нескольких пользователей.

Постановка задачи

Целью курсовой работы является разработка клиент-серверного приложения для интернет-магазина велосипедов.

В ходе выполнения работы необходимо решить следующие задачи:

- спроектировать структуру базы данных PostgreSQL;
- реализовать серверную часть на Python с использованием Flask;
- разработать клиентскую часть на HTML, CSS и JavaScript;
- реализовать CRUD-операции для товаров и заказов;
- обеспечить аутентификацию пользователей;
- реализовать представления, функции, хранимые процедуры и триггеры на стороне БД.

Проектирование архитектуры приложения

Диаграммы проектирования

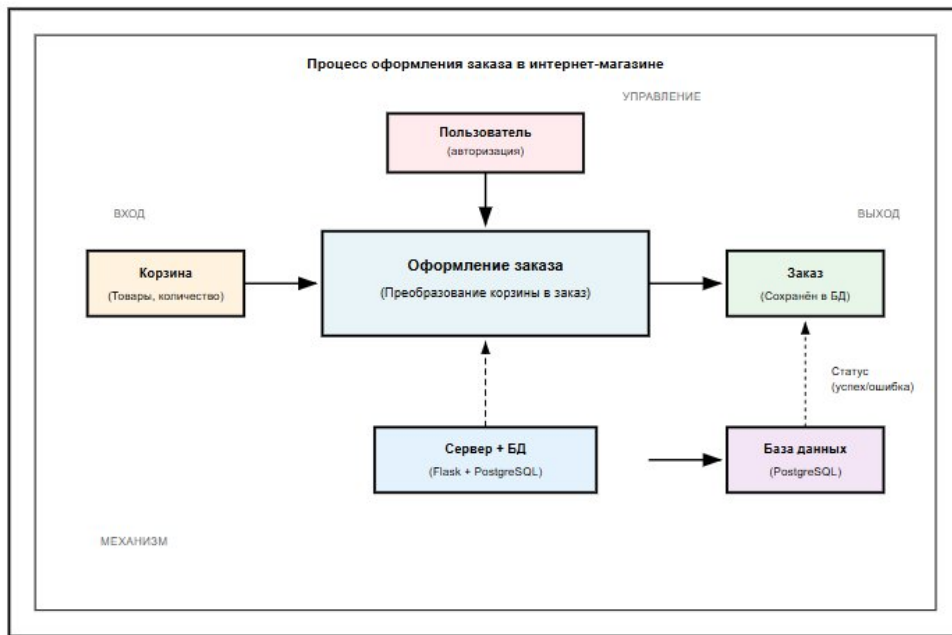


Рисунок 2 — IDEF0-диаграмма процесса оформления заказа

На IDEF0-диаграмме изображён процесс оформления заказа в интернет-магазине. Входом процесса является корзина с выбранными товарами. Управление осуществляет пользователь (авторизация). Механизмом выступают серверная часть (Flask) и база данных (PostgreSQL). Выходом процесса является сохранённый в БД заказ.

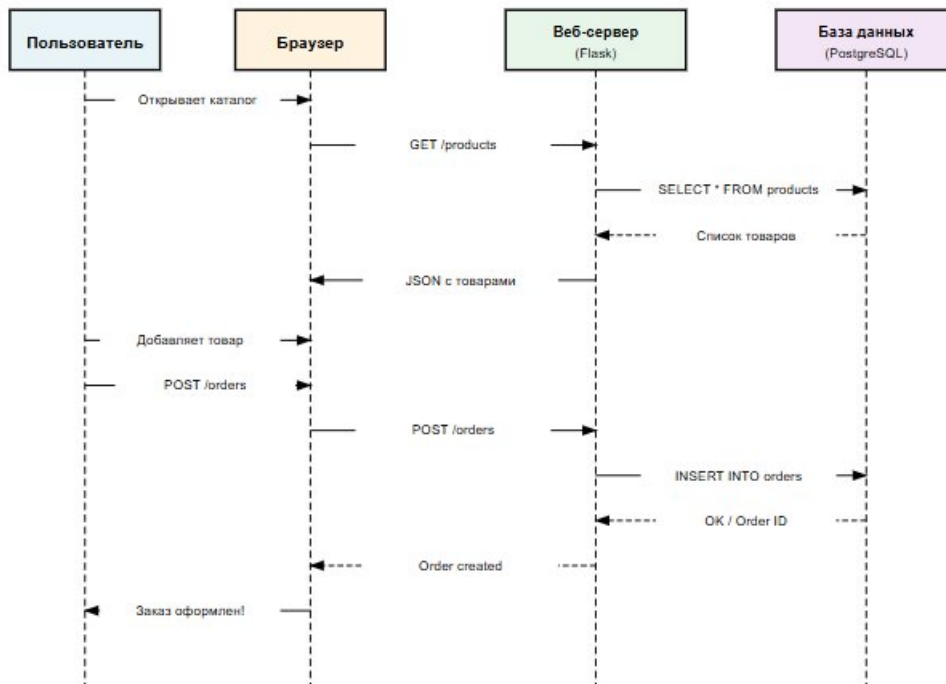


Рисунок 3 — UML-диаграмма последовательности действий пользователя

UML-диаграмма последовательности показывает взаимодействие между пользователем, браузером, сервером и базой данных при оформлении заказа. Пользователь инициирует действие (добавление товара, оформление заказа), браузер отправляет HTTP-запросы, сервер обрабатывает их и выполняет SQL-запросы к БД.

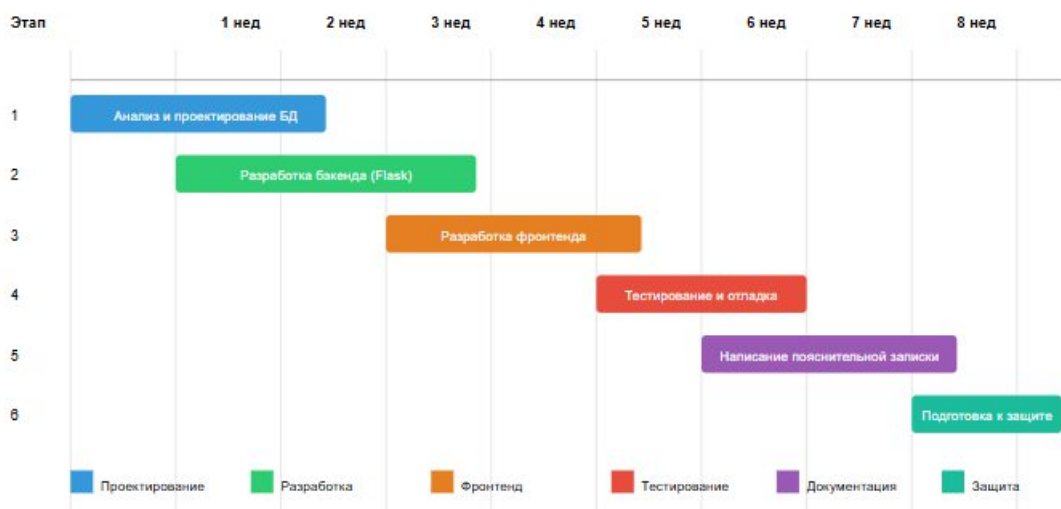


Рисунок 4 — Диаграмма Ганта этапов разработки

Диаграмма Ганта отображает план-график выполнения этапов разработки. Проект разбит на 6 этапов: анализ и проектирование БД (1 неделя), разработка бэкенда (2 недели), разработка фронтенда (2 недели), тестирование (1.5 недели), написание отчёта (1.5 недели), подготовка к защите (1 неделя).

Архитектура приложения

Разработанное приложение построено по классической клиент-серверной архитектуре. Взаимодействие между компонентами осуществляется по протоколу HTTP с обменом данными в формате JSON.

Серверная часть (Backend): Python 3.12, Flask, psycopg2-binary

Клиентская часть (Frontend): HTML5, CSS3, JavaScript (ES6+)

Проектирование базы данных

Структура базы данных

База данных содержит 4 основные таблицы: users, products, orders, order_items.

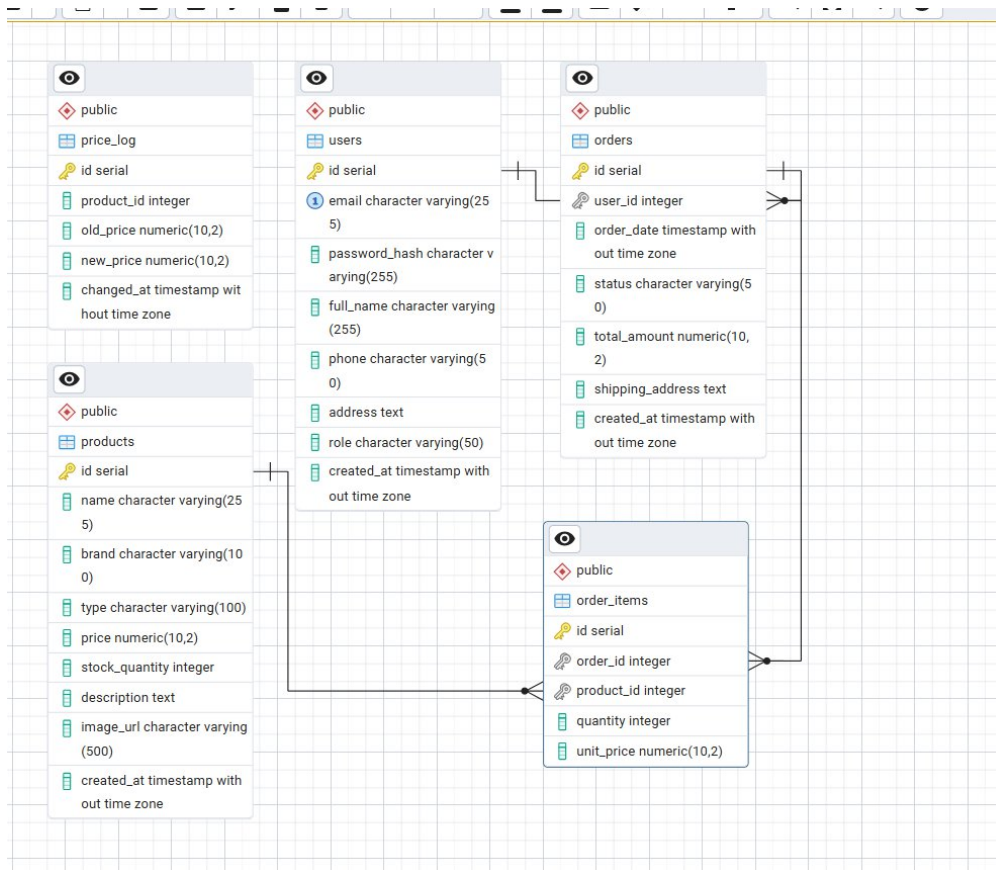


Рисунок 5 — ER Диаграмма

Представления

Созданы три представления: user_order_summary, popular_products, monthly_sales.

Query

Query History

1

SELECT * FROM popular_products

Data Output

Messages

Notifications

SQL

Showing

	id integer	name character varying (255)	brand character varying (100)	price numeric (10,2)	total_sold bigint	stock_quantity integer
1	4	Trek Marlin 7	Trek	65990.00	5	0
2	1	Stels Navigator 700	Stels	38000.00	2	14
3	3	Trek Marlin 5	Trek	49990.00	1	8
4	10	Specialized Rockhopper	Specialized	69990.00	0	4
5	15	Trek FX 2 Disc	Trek	47990.00	0	9
6	6	Cube Aim Pro	Cube	45990.00	0	12
7	12	Scott Aspect 950	Scott	39990.00	0	14
8	19	Giant Escape 2	Giant	38990.00	0	18

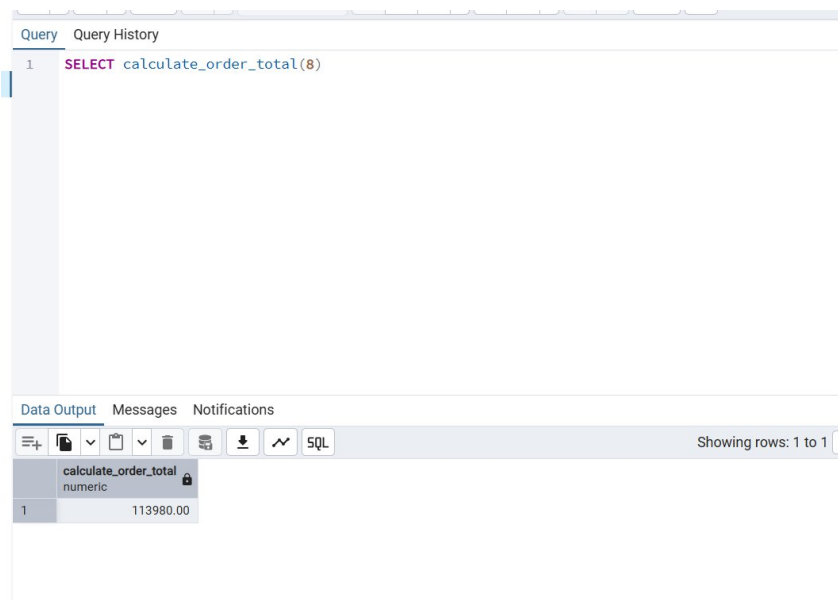


Рисунок 7 — Использование функции

Хранимые процедуры

- `apply_discount(product_type, percent)` — применение скидки;
- `advance_order_status(order_id)` — смена статуса заказа;
- `archive_old_orders(days)` — архивация старых заказов.

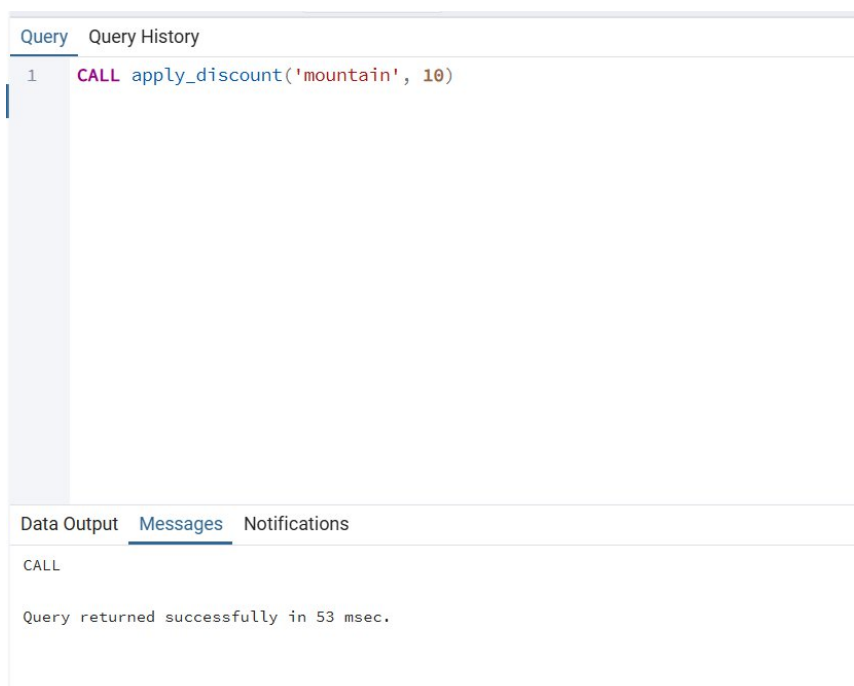


Рисунок 8 — Использование процедуры

Триггеры:

- `trigger_check_stock` — проверка наличия товара;
- `trigger_price_log` — логирование изменения цен;

- trigger_update_updated_at — автообновление updated_at.

Query

Query History

1

SELECT * FROM price_log

Data Output

Messages

Notifications

Рисунок 9 — Триггер

Реализация серверной части

Выбор технологий

Серверная часть реализована на Python с использованием микрофреймворка Flask

Основные эндпоинты API

Метод	Эндпоинт	Описание
GET	/products	Список товаров с фильтрацией
GET	/products/{id}	Товар по ID
POST	/register	Регистрация
POST	/login	Авторизация
GET	/orders	Список заказов
POST	/orders	Создание заказа

GET /products — получение списка товаров

Назначение: возвращает список всех товаров из базы данных с возможностью фильтрации.

HTTP-метод: GET

Параметры запроса (опционально):

type — фильтр по типу велосипеда (mountain, hybrid, city)

brand — фильтр по бренду (Stels, Trek, Merida и др.)

min_price — минимальная цена

max_price — максимальная цена

Пример

запроса:

GET http://localhost:8000/products?type=mountain&min_price=30000

Что происходит внутри функции:

- Функция получает параметры из URL через `request.args.get()`
- Формирует SQL-запрос с добавлением условий WHERE в зависимости от переданных параметров

- [illegible]

5.2.2 GET /products/{id} — получение одного товара

HTTP-метод: GET

Пример запроса:

Что происходит внутри функции:

- 16

[illegible]

Рисунок 11 — Получение одного товара

POST /register — регистрация нового пользователя

Назначение: создаёт нового пользователя в системе.

HTTP-метод: POST

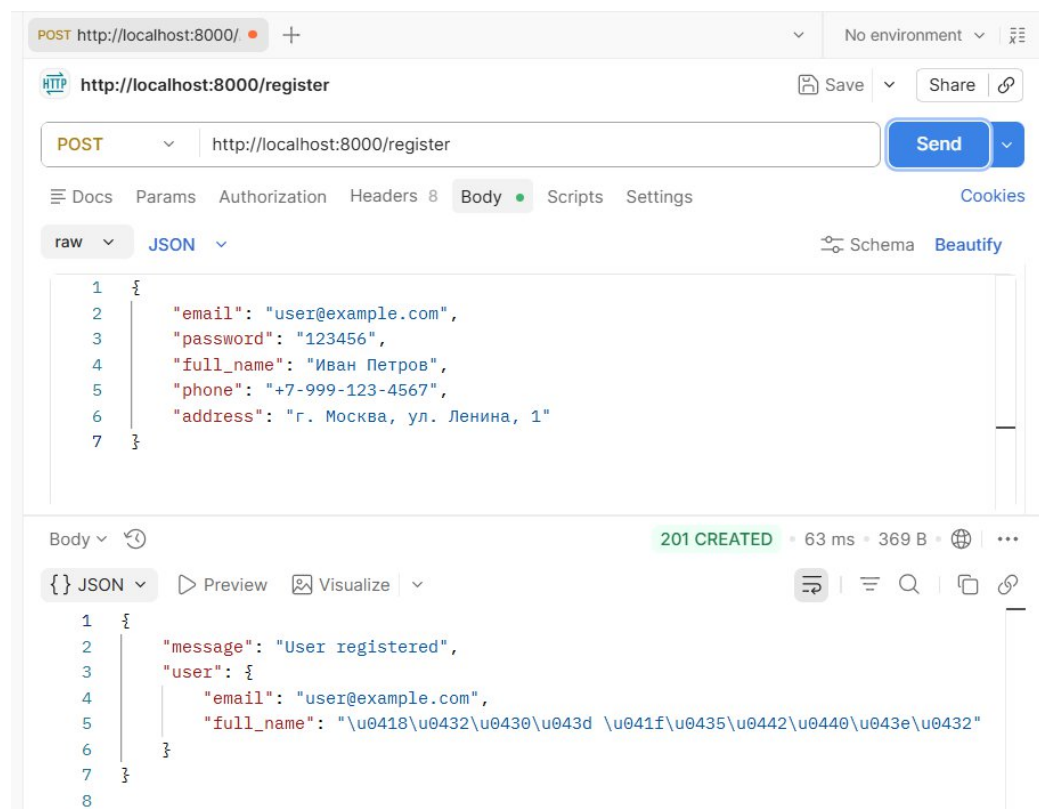


Рисунок 12 — Успешная попытка регистрации

Что происходит внутри функции:

- Функция получает JSON из тела запроса через `request.json`
- Проверяет, существует ли пользователь с таким email (выполняет `SELECT` по email)
- Если email уже зарегистрирован — возвращает ошибку 400

- Если email свободен — хеширует пароль с помощью passlib (алгоритм bcrypt)
- Выполняет SQL-запрос INSERT INTO users
- Сохраняет пользователя в базу данных
- Возвращает сообщение об успешной регистрации

POST /login — авторизация пользователя

Назначение: проверяет учётные данные пользователя и выполняет вход в систему.

HTTP-метод: POST

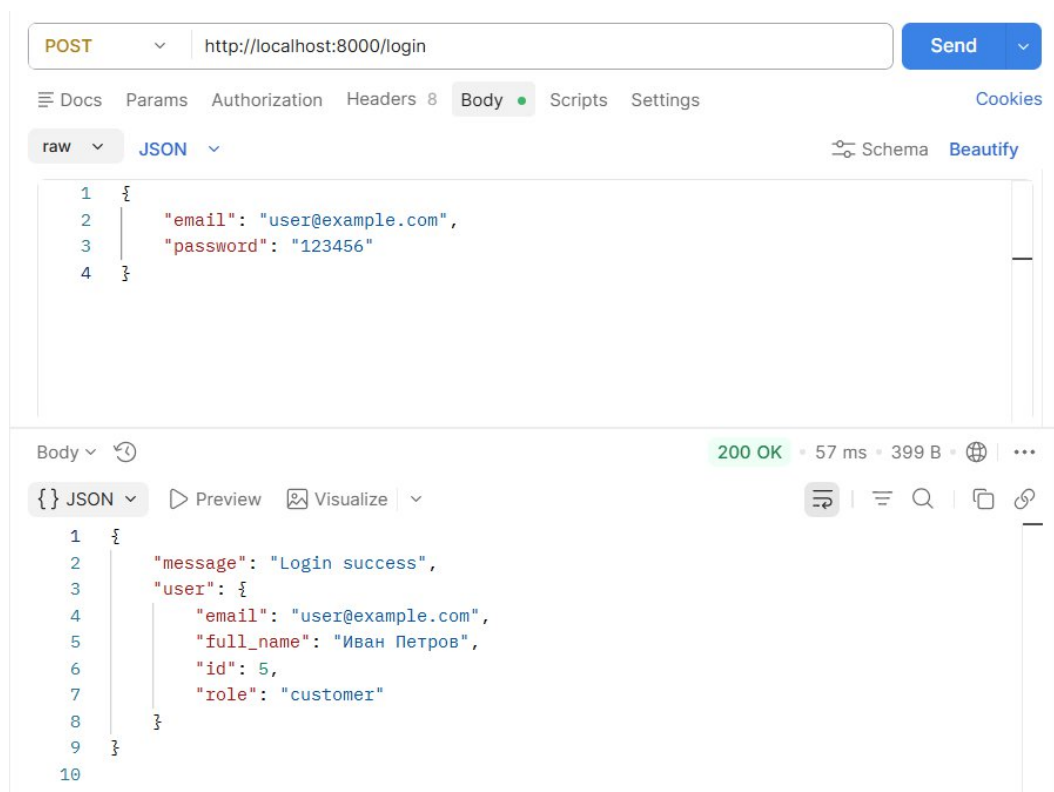


Рисунок 13 — Успешная попытка логирования

Что происходит внутри функции:

- Функция получает email и пароль из тела запроса
- Выполняет SQL-запрос SELECT * FROM users WHERE email = %s
- Если пользователь не найден — возвращает ошибку 401
- Если пользователь найден — сравнивает введенный пароль с хешем из базы данных

- Если пароль не совпадает — возвращает ошибку 401
- Если пароль верный — возвращает данные пользователя (id, email, full_name, role)

GET /orders — получение списка заказов пользователя

Назначение: возвращает все заказы, сделанные конкретным пользователем.

HTTP-метод: GET

Параметры: user_id — идентификатор пользователя (передаётся в строке запроса)

Пример запроса:

GET http://localhost:8000/orders?user_id=1

Что происходит внутри функции:

- Функция получает user_id из параметров URL
- Выполняет SQL-запрос с JOIN для получения заказов и связанных с ними товаров
- Формирует вложенную структуру JSON (каждый заказ содержит массив позиций)
- Возвращает список заказов клиенту

```
[
  {
    "created_at": "Mon, 25 May 2026 22:59:29 GMT",
    "id": 1,
    "items": [
      {
        "price": null,
        "product_name": null,
        "quantity": null
      }
    ],
    "order_date": "Mon, 25 May 2026 22:59:29 GMT",
    "shipping_address": "\u041c\u043e\u0441\u043a\u0430, \u0432\u043e\u043b\u043e\u0433\u043e\u0432\u0430\u0439, \u0432\u043e\u043b\u043e\u0433\u043e\u0432\u0430\u0439, 1",
    "status": "pending",
    "total_amount": "0.00",
    "user_id": 1
  },
  {
    "created_at": "Mon, 25 May 2026 22:59:29 GMT",
    "id": 2,
    "items": [
      {
        "price": null,
        "product_name": null,
        "quantity": null
      }
    ],
    "order_date": "Mon, 25 May 2026 22:59:29 GMT",
    "shipping_address": "\u041c\u043e\u0441\u043a\u0430, \u0432\u043e\u043b\u043e\u0433\u043e\u0432\u0430\u0439, \u0432\u043e\u043b\u043e\u0433\u043e\u0432\u0430\u0439, 1",
    "status": "pending",
    "total_amount": "0.00",
    "user_id": 1
  },
  {
    "created_at": "Mon, 25 May 2026 23:43:13 GMT",
    "id": 8,
    "items": [
      {
        "price": 35990.0,
        "product_name": "Stels Navigator 700",
        "quantity": 2
      },
      {
        "price": 42000.0,
        "product_name": "Trek Marlin 5",
        "quantity": 1
      }
    ],
    "order_date": "Fri, 15 May 2026 14:30:00 GMT",
    "shipping_address": "\u041c\u043e\u0441\u043a\u0430, \u0432\u043e\u043b\u043e\u0433\u043e\u0432\u0430\u0439, \u0432\u043e\u043b\u043e\u0433\u043e\u0432\u0430\u0439, 1",
    "status": "delivered",
    "total_amount": "113990.00",
    "user_id": 1
  }
]
```

Рисунок 14 — Получение списка заказов пользователя

POST /orders — создание нового заказа

Назначение: оформляет новый заказ на основе содержимого корзины.

HTTP-метод: POST

Параметры: `user_id` — идентификатор пользователя (передается в строке запроса)

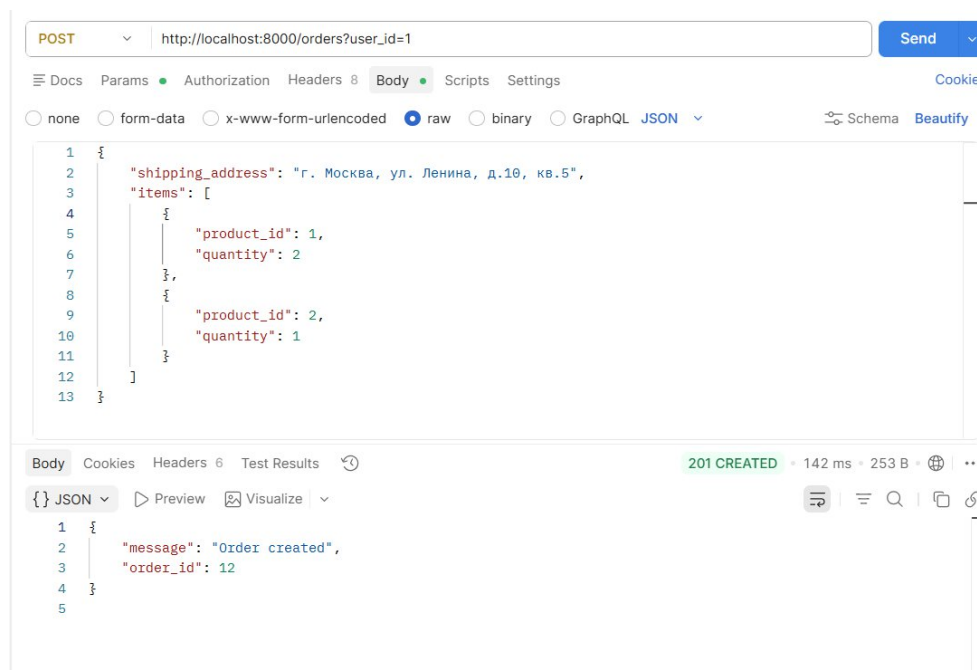


Рисунок 15 — Создание заказа

Что происходит внутри функции:

- Функция получает `user_id` из параметров URL и данные из тела запроса
- Начинает транзакцию (группа операций, которые выполняются вместе)
- Для каждого товара в корзине:
- Проверяет, существует ли товар
- Проверяет, достаточно ли товара на складе
- Вычисляет цену товара
- Создаёт запись в таблице `orders`
- Для каждого товара создаёт запись в таблице `order_items`
- Вызывает функцию `update_stock()` для уменьшения количества на складе
- Вызывает функцию `calculate_order_total()` для расчёта итоговой суммы
- Фиксирует транзакцию (COMMIT)
- Возвращает ID созданного заказа

```

@app.route('/products', methods=['GET'])
def get_products():
    # Получаем параметры фильтрации из URL
    product_type = request.args.get('type')
    brand = request.args.get('brand')
    min_price = request.args.get('min_price')
    max_price = request.args.get('max_price')

    conn = get_db_connection()
    cur = conn.cursor(cursor_factory=psycopg2.extras.RealDictCursor)

    # Базовый запрос
    query = "SELECT * FROM products WHERE 1=1"
    params = []

    # Добавляем фильтры
    if product_type:
        query += " AND type = %s"
        params.append(product_type)
    if brand:
        query += " AND brand = %s"
        params.append(brand)
    if min_price:
        query += " AND price >= %s"
        params.append(float(min_price))
    if max_price:
        query += " AND price <= %s"
        params.append(float(max_price))

    query += " ORDER BY id"

    cur.execute(query, params)
    products = cur.fetchall()
    cur.close()
    conn.close()
    return jsonify(products)

```

Рисунок 16 — Код функции `get_products()`

Листинг функции `get_products()` показывает реализацию эндпоинта для получения списка товаров.

Функция принимает из URL параметры фильтрации (`type`, `brand`, `min_price`, `max_price`),

формирует динамический SQL-запрос и возвращает результат в формате JSON.

Особенность реализации — использование параметризованных запросов (`%s`) для защиты от SQL-инъекций.

```

@app.route('/orders', methods=['POST'])
def create_order():
    data = request.json
    user_id = request.args.get('user_id')
    conn = get_db_connection()
    cur = conn.cursor()
    try:
        # Создаем заказ
        cur.execute(
            "INSERT INTO orders (user_id, shipping_address) VALUES (%s, %s) RETURNING id",
            (user_id, data['shipping_address'])
        )
        order_id = cur.fetchone()[0]

        # Добавляем позиции
        for item in data['items']:
            cur.execute("SELECT price FROM products WHERE id = %s", (item['product_id'],))
            price = cur.fetchone()[0]
            cur.execute(
                "INSERT INTO order_items (order_id, product_id, quantity, unit_price) VALUES (%s, %s, %s, %s)",
                (order_id, item['product_id'], item['quantity'], price)
            )
            # Вызываем функцию обновления остатков (PostgreSQL)
            cur.execute("SELECT update_stock(%s, %s)", (item['product_id'], item['quantity']))

        # Считаем итог
        cur.execute("SELECT calculate_order_total(%s)", (order_id,))
        conn.commit()
        return jsonify({"message": "Order created", "order_id": order_id}), 201
    except Exception as e:
        conn.rollback()
        return jsonify({"error": str(e)}), 400
    finally:
        cur.close()
        conn.close()

```

Рисунок 17 — Код функции `create_order()`

Листинг функции `create_order()` демонстрирует обработку POST-запроса на создание заказа.

Функция получает `user_id` из URL и JSON с адресом доставки и товарами из тела запроса.

Внутри функции запускается транзакция: проверяется наличие товаров на складе,

создаётся запись в таблице `orders`, добавляются позиции в `order_items`, обновляются остатки через функцию `update_stock()` и рассчитывается сумма через `calculate_order_total()`.

Логирование и безопасность

В целях обеспечения безопасности приложения реализованы следующие механизмы:

Хеширование паролей: пароли пользователей не хранятся в открытом виде. При регистрации пароль хешируется с использованием библиотеки `passlib` (алгоритм `bcrypt`), в базу данных сохраняется только хеш. При авторизации введённый пароль хешируется и сравнивается с сохранённым хешем.

Защита от SQL-инъекций: все SQL-запросы используют параметризованные выражения (плейсхолдеры `%s`). Это исключает возможность внедрения злоумышленником вредоносного SQL-кода через пользовательский ввод.

Логирование действий: в перспективе планируется реализовать логирование всех операций с заказами (кто, когда, какой заказ создал/изменил). Это позволит проводить аудит безопасности и отслеживать подозрительную активность.

Реализация клиентской части

Структура клиентского приложения

Клиентская часть состоит из 5 HTML-страниц: index.html, catalog.html, product.html, cart.html, orders.html.

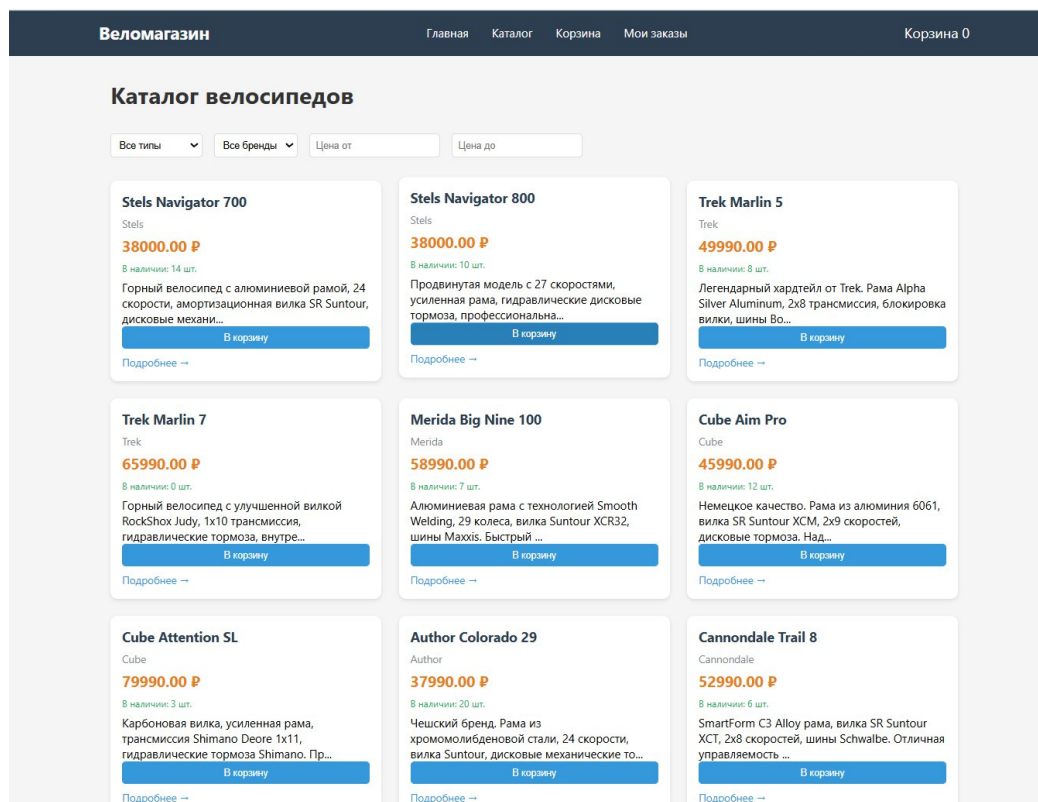


Рисунок 18 — Каталог

Функции взаимодействия с API

```
async function apiRequest(endpoint, method = 'GET', body = null) {
  const options = {
    method: method,
    headers: {
      'Content-Type': 'application/json',
    }
  };

  if (body) {
    options.body = JSON.stringify(body);
  }

  try {
    const response = await fetch(`${API_URL}${endpoint}`, options);
    const data = await response.json();

    if (!response.ok) {
      throw new Error(data.detail || 'Ошибка запроса');
    }

    return data;
  } catch (error) {
    console.error('API Error:', error);
    throw error;
  }
}
```

Рисунок 19— Реализация функции *apiRequest()*

Функция *apiRequest()* является универсальной обёрткой для отправки HTTP-запросов к серверу.

Она принимает эндпоинт, метод и тело запроса, автоматически устанавливает заголовок

Content-Type: application/json, обрабатывает ответ и возвращает JSON.

В случае ошибки функция выбрасывает исключение с понятным сообщением.

```

async function loadProducts() {
  const productList = document.getElementById('productsList');
  if (!productList) return;

  productList.innerHTML = '<div class="loading">Загрузка...</div>';

  try {
    let url = '/products';
    const params = new URLSearchParams();

    const type = document.getElementById('filterType')?.value;
    const brand = document.getElementById('filterBrand')?.value;
    const minPrice = document.getElementById('minPrice')?.value;
    const maxPrice = document.getElementById('maxPrice')?.value;

    if (type && type !== '') params.append('type', type);
    if (brand && brand !== '') params.append('brand', brand);
    if (minPrice) params.append('min_price', minPrice);
    if (maxPrice) params.append('max_price', maxPrice);

    if (params.toString()) {
      url += '?' + params.toString();
    }

    const products = await apiRequest(url);

    if (products.length === 0) {
      productList.innerHTML = '<div class="loading">Товары не найдены</div>';
      return;
    }

    productList.innerHTML = products.map(product => `
      <div class="product-card">
        <a href="product.html?id=${product.id}" style="text-decoration: none; color: inherit;">
          <h3>${product.name}</h3>
        </a>
        <div class="brand">${product.brand}</div>
        <div class="price">${product.price.toLocaleString()} Р</div>
        <div class="stock">В наличии: ${product.stock_quantity} шт.</div>
        <p>${(product.description || '').substring(0, 100)}${(product.description || '').length > 100 ? '...' : ''}</p>
        <button onclick="event.stopPropagation(); addToCart(${product.id}, '${product.name}', ${product.price})">
          В корзину
        </button>
        <a href="product.html?id=${product.id}" class="details-link">Подробнее </a>
      </div>
    `).join('');

  } catch (error) {
    console.error('Ошибка:', error);
    productList.innerHTML = '<div class="loading">Ошибка загрузки товаров</div>';
  }
}

```

Рисунок 20— Реализация функции *loadProducts()*

Функция *loadProducts()* отвечает за загрузку и отображение товаров на странице каталога.

Она собирает параметры фильтрации из выпадающих списков и полей ввода,

формирует URL с query-параметрами, вызывает *apiRequest()* и динамически создаёт

HTML-карточки товаров на основе полученных данных.

```

function addToCart(productId, name, price) {
  const existingItem = cart.find(item => item.product_id === productId);

  if (existingItem) {
    existingItem.quantity++;
  } else {
    cart.push({
      product_id: productId,
      name: name,
      price: price,
      quantity: 1
    });
  }

  saveCart();
  alert(`${name} добавлен в корзину!`);
}

function renderCart() {
  const cartItems = document.getElementById('cartItems');
  const totalPriceSpan = document.getElementById('totalPrice');

  if (!cartItems) return;

  if (cart.length === 0) {
    cartItems.innerHTML = '<div class="loading">Корзина пуста</div>';
    if (totalPriceSpan) totalPriceSpan.textContent = '0';
    return;
  }

  let total = 0;

  cartItems.innerHTML = cart.map(item => {
    const itemTotal = item.price * item.quantity;
    total += itemTotal;

    return `
      <div class="cart-item">
        <div class="cart-item-info">
          <h3>${item.name}</h3>
          <div>${item.price.toLocaleString()} Р</div>
        </div>
        <div class="cart-item-quantity">
          <button onclick="updateQuantity(${item.product_id}, -1)">-</button>
          <span>${item.quantity}</span>
          <button onclick="updateQuantity(${item.product_id}, 1)">+</button>
        </div>
        <div class="cart-item-price">${itemTotal.toLocaleString()} Р</div>
        <button class="remove-btn" onclick="removeFromCart(${item.product_id})">Удалить</button>
      </div>
    `;
  }).join('');

  if (totalPriceSpan) totalPriceSpan.textContent = total.toLocaleString();
}

```

Рисунок 21— Реализация функции *addToCart()*

Функция *addToCart()* добавляет товар в корзину. При повторном добавлении того же товара

увеличивается его количество, иначе создаётся новый объект.

Функция *saveCart()* сохраняет корзину в *localStorage*.

Функция *renderCart()* отображает содержимое корзины на странице и подсчитывает итоговую сумму.

Функция *updateQuantity()* изменяет количество товара и перерисовывает корзину.

```

async function checkout() {
  if (!checkAuth()) return;

  if (cart.length === 0) {
    alert('Корзина пуста');
    return;
  }

  const address = prompt('Введите адрес доставки:');
  if (!address) return;

  const items = cart.map(item => ({
    product_id: item.product_id,
    quantity: item.quantity
  }));

  try {
    const result = await apiRequest(`/orders?user_id=${currentUser.id}`, 'POST', {
      shipping_address: address,
      items: items
    });

    alert('Заказ успешно оформлен!');
    cart = [];
    saveCart();
    window.location.href = 'orders.html';
  } catch (error) {
    alert('Ошибка оформления заказа: ' + error.message);
  }
}

```

Рисунок 22— Реализация функции *checkout()*

Функция `checkout()` оформляет заказ. Она проверяет авторизацию пользователя,

запрашивает у пользователя адрес доставки через `prompt()`,

формирует объект с данными заказа и отправляет POST-запрос на сервер.

При успешном ответе корзина очищается, и пользователь перенаправляется на страницу заказов.

Тестирование приложения

Тестирование API проводилось с использованием инструмента Postman.

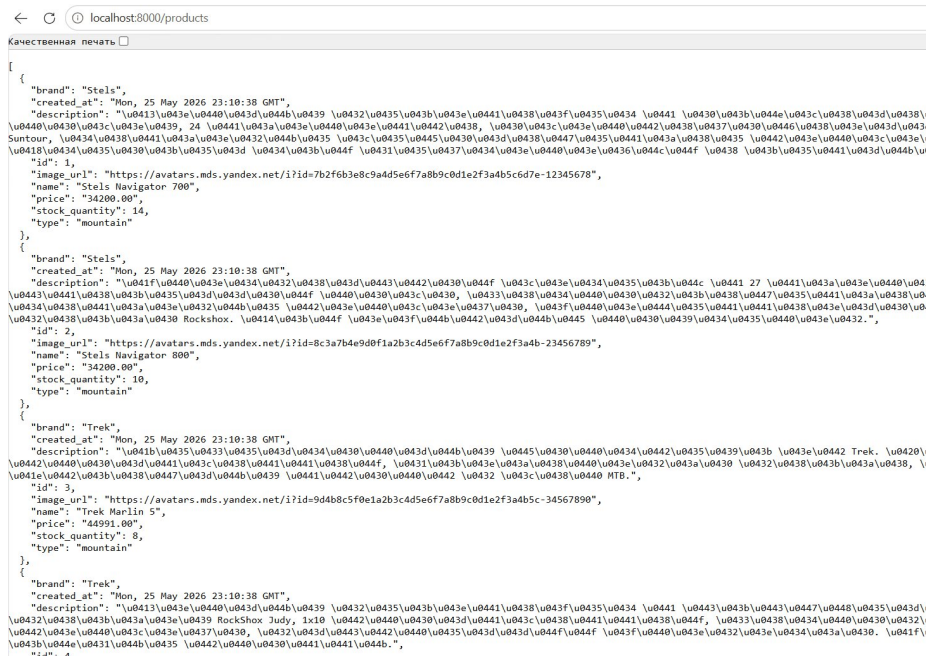


Рисунок 23— Демонстрация GET – запроса

Тестирование GET-запроса на получение списка товаров.

В Postman был отправлен запрос GET `http://localhost:8000/products`.

Сервер вернул HTTP-статус 200 ОК и список всех товаров в формате JSON.

На скриншоте видно, что каждый товар содержит поля id, name, brand, type, price и другие.

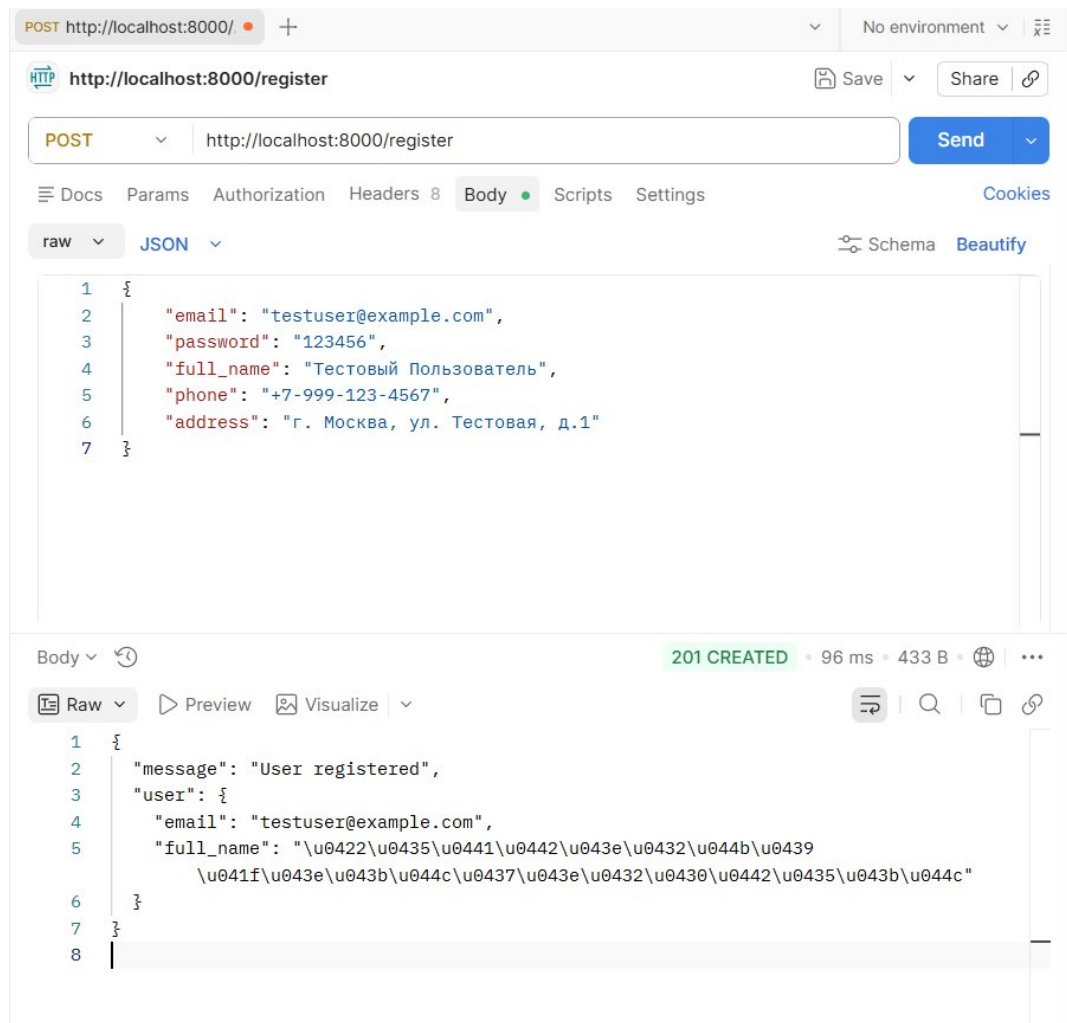


Рисунок 24— Демонстрация POST – запроса

Тестирование регистрации нового пользователя.

Был отправлен POST-запрос на эндпоинт /register с JSON-телом, содержащим email,

пароль, ФИО, телефон и адрес. Сервер успешно создал пользователя и вернул

сообщение "User registered" с HTTP-статусом 201 Created.

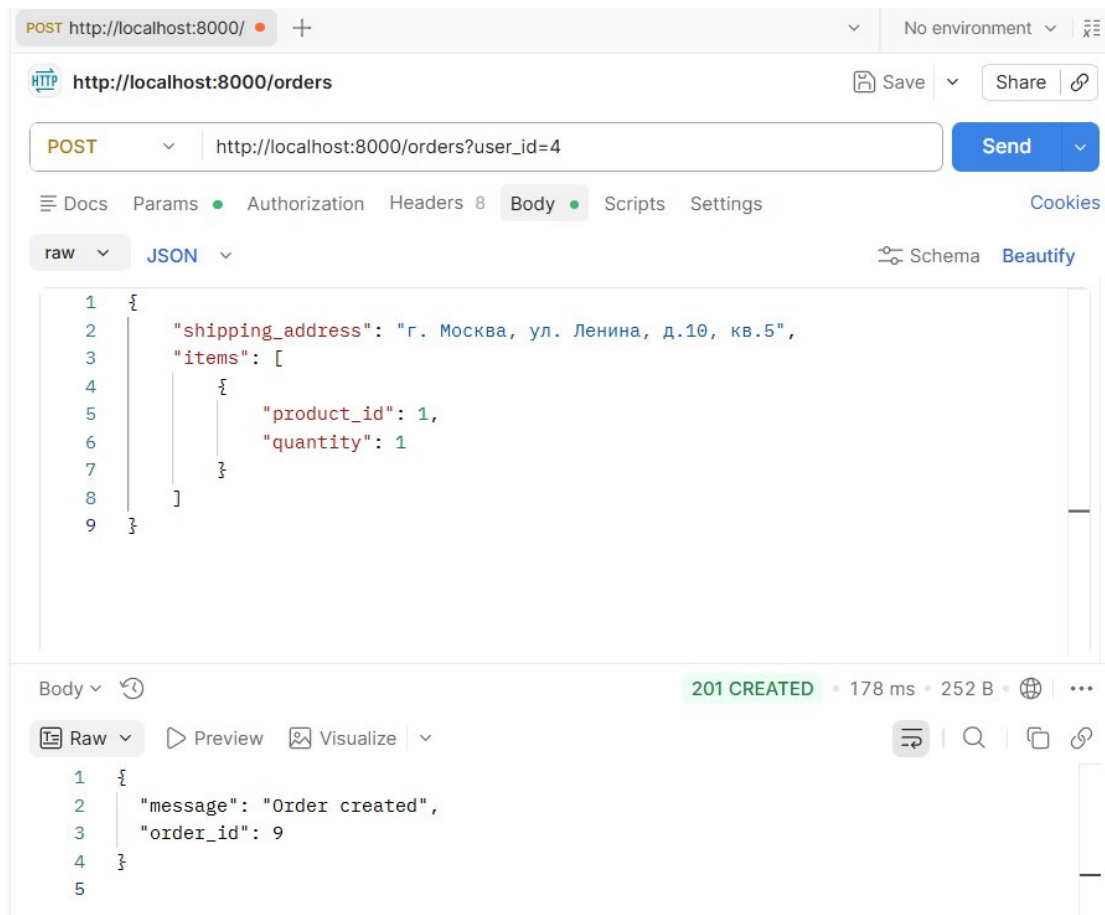


Рисунок 25— Демонстрация *POST* – запроса

Тестирование создания нового заказа.

Был отправлен *POST*-запрос на эндпоинт `/orders` с параметром `?user_id=1`

и *JSON*-телом, содержащим адрес доставки и массив товаров с их количеством.

Сервер создал заказ и вернул сообщение "Order created" с идентификатором заказа.

A screenshot of a web application showing a registration modal window. The modal is titled "Регистрация" (Registration) and has a close button (X) in the top right corner. It contains five input fields: "ФИО" (Full Name), "Email", "Пароль" (Password), "Телефон" (Phone), and "Адрес" (Address). Below the fields is a blue button labeled "Зарегистрироваться" (Register). The background of the website is dark gray, and there is a banner on the right that says "Быстрая доставка По всей России" (Fast delivery Across the whole of Russia).

Рисунок 26— Регистрация

Тестирование регистрации через графический интерфейс.

При нажатии кнопки "Регистрация" открывается модальное окно с полями для ввода

email, пароля, ФИО, телефона и адреса. После заполнения формы и отправки

пользователь получает уведомление об успешной регистрации.

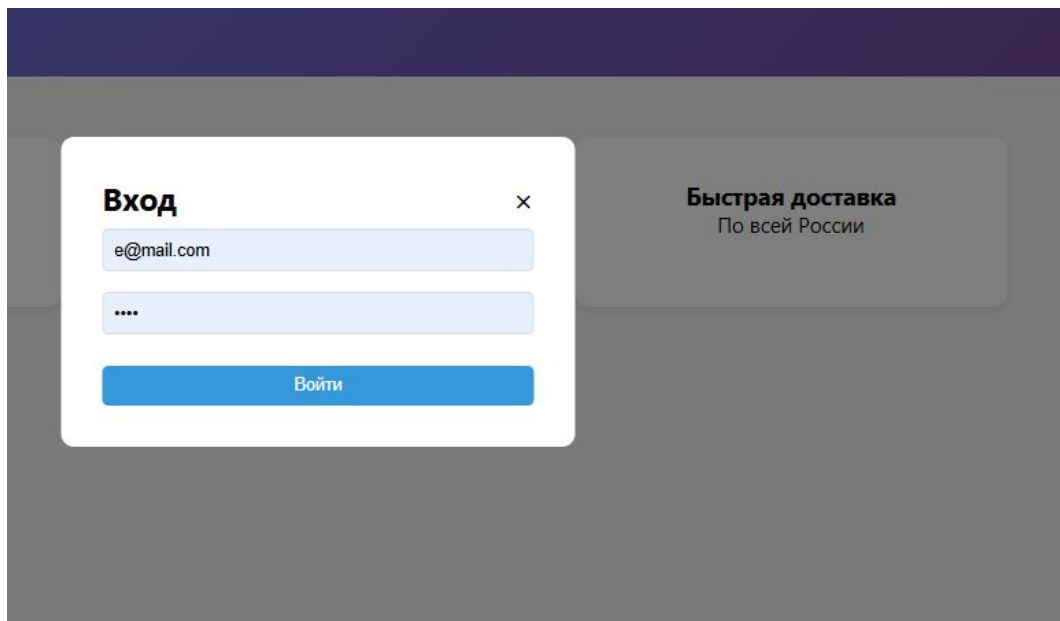


Рисунок 27— Вход

Тестирование авторизации через графический интерфейс.

При нажатии кнопки "Вход" открывается модальное окно с полями для email и пароля.

После успешного входа в правом верхнем углу отображается имя пользователя и кнопка "Выйти".

Заключение

В ходе выполнения курсового проекта была разработана клиент-серверная система для интернет-магазина велосипедов. Все поставленные задачи выполнены.

В результате выполнения курсового проекта получили следующие компетенции:

- Способность анализировать профессиональную информацию;
- Способность использовать методы математического моделирования;
- Способность выполнять работы по созданию информационных систем;
- Способность анализировать процессы и решать нестандартные задачи;
- Способность проектировать и разрабатывать продукцию в условиях неопределённости.

В ходе выполнения курсовой работы реализовано:

- Клиент-серверное приложение для интернет-магазина велосипедов
- База данных PostgreSQL из 4 таблиц, 3 представлений, 3 функций, 3 хранимых процедур
- REST API на Flask с 6 эндпоинтами (GET /products, GET /products/{id}, POST /register, POST /login, GET /orders, POST /orders)
- Фронтенд на HTML/CSS/JS с фильтрацией товаров, корзиной, оформлением заказов
- Аутентификация пользователей (регистрация, вход, выход)
- Хеширование паролей и защита от SQL-инъекций

Список литературы

1. Гриджук, Д. FastAPI. Создание высокопроизводительных веб-приложений на Python / Д. Гриджук. — Санкт-Петербург: Питер, 2023. — 352 с.
2. Любанович, Б. Простой Python. Современный стиль программирования / Б. Любанович. — Санкт-Петербург: Питер, 2022. — 480 с.
3. Дударев, В.А. Базы данных: учебное пособие / В.А. Дударев. — Москва: МИСИС, 2023. — 156 с.
4. Мартин, Р. Чистая архитектура. Искусство разработки программного обеспечения / Р. Мартин. — Санкт-Петербург: Питер, 2018. — 352 с.
5. Клеппман, М. Высоконагруженные приложения. Программирование, масштабирование, поддержка / М. Клеппман. — Санкт-Петербург: Питер, 2018. — 640 с.
6. Ссылка на репозиторий с проектом — [Nes776/-veloshop-course-work](https://github.com/Nes776/veloshop-course-work)